TerraMind on SmallMinesDS: Final Report

Oct 26, 2026 · Asım Emre Arslan

Summary

We compared a frozen, pretrained TerraMind v1 tiny encoder with the same encoder trained from scratch for mine segmentation on SmallMinesDS 2022, using 1 to 1,491 (all) training images, in two experiments. **Experiment 1** trained every run for a fixed 50 epochs under four conditions (each encoder with and without D4 augmentation, 196 runs): augmentation improved the frozen encoder in every seed (+0.02 to +0.04 IoU) but had no consistent effect on the trainable one, so only the augmented conditions were kept. **Experiment 2** replaced the fixed budget with early stopping on test IoU and recorded the cost of every run (98 runs). In both experiments the frozen pretrained encoder gives the higher IoU at every N ⩾ 10 in every seed (all images: 0.745 vs 0.673 in Experiment 1, 0.723 vs 0.642 in Experiment 2), and with 10–20 images it matches what the trainable encoder reaches with all 1,491. It is also cheaper to train: 2.4 vs 5.5 GFLOP per training sample, and with all images 0.095 vs 0.173 GPU hours until early stopping.

Setup (both experiments)

Item	Setting
Data	SmallMinesDS 2022, 128 × 128 px patches; official split: 1,491 training, 644 test
Inputs	Sentinel-2 L2A (10 bands) + Sentinel-1 RTC (VV, VH, in dB); DEM not used
Model	TerraMind v1 tiny encoder (5.9 M parameters) + shallow UNet decoder (1.0 M), built with TerraTorch
Encoder conditions	Pretrained, frozen (only the decoder learns) / random init, trainable
Augmentation	D4: random rotation by 0/90/180/270°, with or without mirroring
Training images N	1, 3, 5, 10, 20, 50, 100, 250 (5 seeds); 500, 1000, all = 1,491 (3 seeds)
Optimiser	AdamW, weight decay 0.01, batch size min(8, N)

Item	Setting
Metric	Mine-class IoU on the 644 test patches; mean ± std over seeds
Hardware	NVIDIA GeForce RTX 3080 Laptop GPU (8 GB)

Within a seed, the conditions use the same training images, decoder initialisation and batch order, so they are directly comparable. Code, results and reproduction steps: github.com/asimarslan11/SmallMinesDS-Experiment-Project.

Experiment 1: fixed training budget, four conditions

Every run trained for exactly 50 epochs with a cosine learning rate from 5e-3 to 1e-5, and the final model was evaluated; 4 conditions × 49 (seed, N) pairs = 196 runs, 5.0 GPU hours in total. The frozen pretrained encoder is ahead at every N ⩾ 10 in every seed, and augmentation adds +0.02 to +0.04 IoU to it at every N ⩾ 3 (5/5 seeds), while for the trainable encoder the effect changes sign with N. With 10 images, the frozen encoder with augmentation (0.591) matches the trainable encoder with 250 images (0.584–0.605).

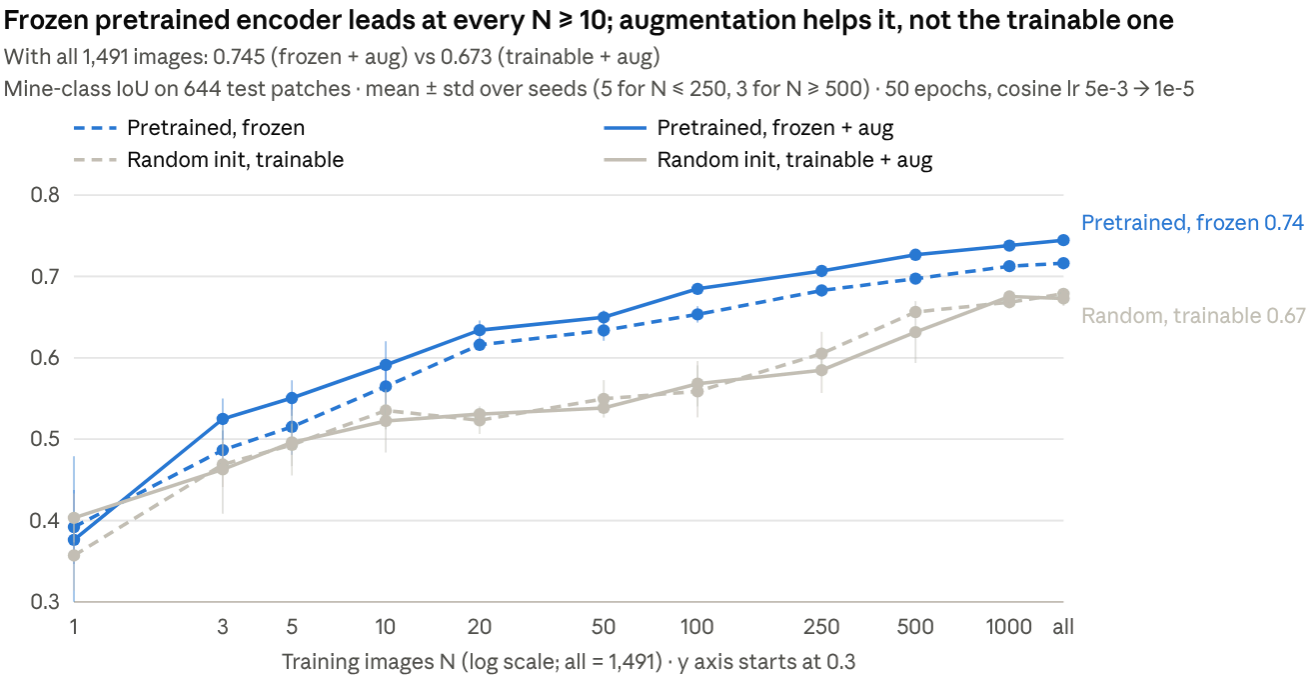


Figure 1. Experiment 1: mine-class IoU on the test set against the number of training images, mean ± std over seeds

Table 1. Experiment 1 — mine-class IoU on the test set (mean ± std over seeds).

N	seeds	Pretrained, frozen	Pretrained, frozen + aug	Random init, trainable	Random init, trainable + aug
1	5	0.392 ± 0.046	0.376 ± 0.103	0.357 ± 0.043	0.403 ± 0.030
3	5	0.487 ± 0.024	0.525 ± 0.025	0.469 ± 0.028	0.463 ± 0.054
5	5	0.515 ± 0.034	0.551 ± 0.022	0.493 ± 0.026	0.496 ± 0.040
10	5	0.565 ± 0.023	0.591 ± 0.029	0.535 ± 0.023	0.522 ± 0.039
20	5	0.616 ± 0.007	0.634 ± 0.012	0.523 ± 0.017	0.531 ± 0.007
50	5	0.633 ± 0.013	0.650 ± 0.008	0.550 ± 0.023	0.538 ± 0.005
100	5	0.653 ± 0.010	0.685 ± 0.007	0.559 ± 0.032	0.568 ± 0.028
250	5	0.683 ± 0.004	0.707 ± 0.002	0.605 ± 0.027	0.584 ± 0.028
500	3	0.697 ± 0.003	0.726 ± 0.003	0.656 ± 0.003	0.632 ± 0.038
1000	3	0.712 ± 0.003	0.738 ± 0.002	0.668 ± 0.006	0.675 ± 0.002
all (1,491)	3	0.716 ± 0.003	0.745 ± 0.003	0.679 ± 0.004	0.673 ± 0.009

Table 2. Experiment 1 — IoU gain from D4 augmentation (augmented minus unaugmented run with the same seed and training images; mean ± std, in brackets the seeds that improved).

N	Pretrained, frozen	Random init, trainable
1	−0.016 ± 0.073 (3/5)	+0.046 ± 0.044 (4/5)
3	+0.038 ± 0.016 (5/5)	−0.006 ± 0.045 (2/5)
5	+0.035 ± 0.027 (5/5)	+0.003 ± 0.034 (3/5)
10	+0.027 ± 0.010 (5/5)	−0.013 ± 0.024 (2/5)
20	+0.018 ± 0.007 (5/5)	+0.008 ± 0.017 (3/5)
50	+0.016 ± 0.005 (5/5)	−0.011 ± 0.024 (2/5)
100	+0.031 ± 0.007 (5/5)	+0.010 ± 0.007 (5/5)
250	+0.024 ± 0.003 (5/5)	−0.021 ± 0.034 (2/5)
500	+0.029 ± 0.002 (3/3)	−0.025 ± 0.036 (1/3)
1000	+0.025 ± 0.002 (3/3)	+0.007 ± 0.008 (2/3)
all (1,491)	+0.029 ± 0.005 (3/3)	−0.006 ± 0.014 (2/3)

Experiment 2: early stopping and training cost

Only the two augmented conditions were run (98 runs, 2.5 GPU hours in total). Instead of a fixed budget, each run trained until its test IoU stopped improving: a check every 5 epochs for N ⩽ 100 and after every epoch for N > 100; training stopped once 20 epochs passed without a gain above 10^{-3} over the best check; the learning rate started at 5e-3 and was halved at every non-improving check (minimum 1e-5); the model at the best check is reported. Steps, FLOPs and GPU hours were recorded for every run.

The frozen pretrained encoder is ahead at every N ⩾ 5 in every seed, 0.723 vs 0.642 with all images, and with 20 images (0.651) it already exceeds the trainable encoder with all 1,491. It costs 2.4 GFLOP per training sample against 5.5 for the trainable encoder, and with all images it stops after 0.095 GPU hours (185 TFLOP) against 0.173 (417 TFLOP). From N = 250 the learning rate is halved after every non-improving epoch rather than every 5 epochs, so it decays faster; this is why the frozen encoder's IoU at N = 250 (0.653) sits below N = 100 (0.694). Both encoders follow the same rule, so their comparison is unaffected.

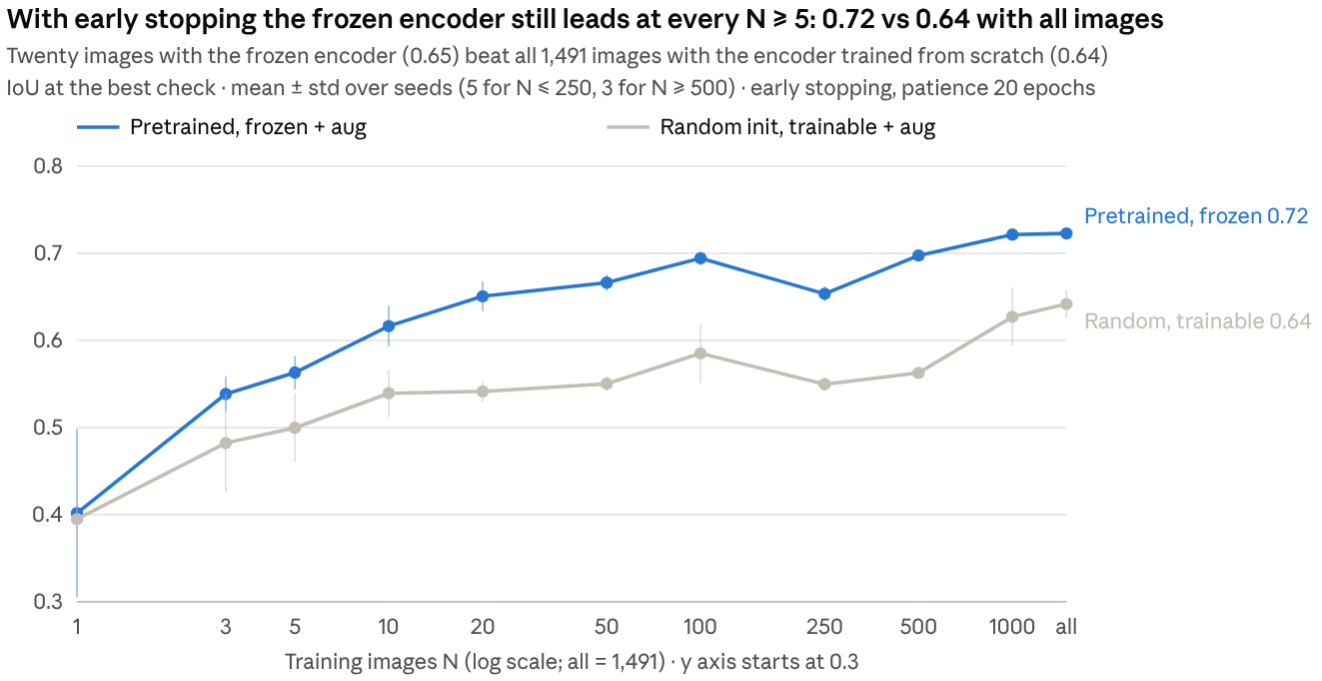


Figure 2. Experiment 2: mine-class IoU (best check) against the number of training images, mean ± std over seeds

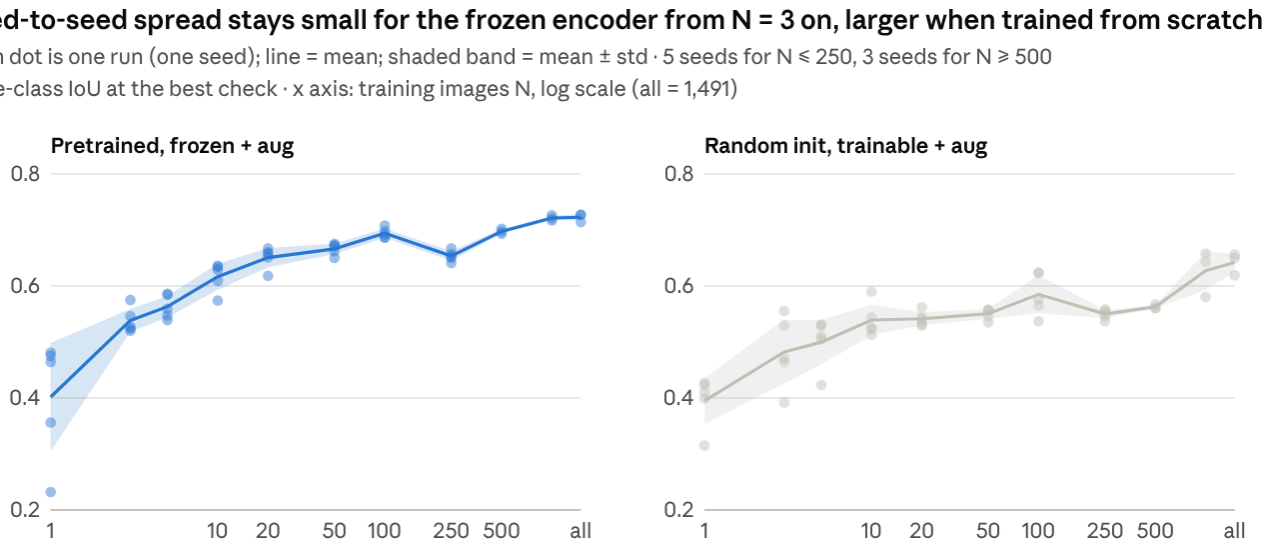


Figure 3. Experiment 2: seed-to-seed variance of IoU — every run as a dot, mean as a line, mean ± std as a shaded band

Table 3. Experiment 2 — mine-class IoU on the test set (best check; mean ± std over seeds).

N	seeds	Pretrained, frozen + aug	Random init, trainable + aug
1	5	0.402 ± 0.096	0.395 ± 0.041
3	5	0.539 ± 0.020	0.482 ± 0.057
5	5	0.563 ± 0.019	0.500 ± 0.040
10	5	0.616 ± 0.023	0.539 ± 0.027
20	5	0.651 ± 0.017	0.541 ± 0.012
50	5	0.666 ± 0.009	0.550 ± 0.009
100	5	0.694 ± 0.008	0.585 ± 0.034
250	5	0.653 ± 0.008	0.550 ± 0.008
500	3	0.698 ± 0.004	0.563 ± 0.003
1000	3	0.722 ± 0.004	0.627 ± 0.034
all (1,491)	3	0.723 ± 0.006	0.642 ± 0.016

Table 4. Experiment 2 — training cost until early stopping, pretrained, frozen + aug (mean ± std over seeds).

N	best epoch	stop epoch	steps	GPU hours	total TFLOP
1	43.0 ± 28.0	63.0 ± 28.0	63 ± 28	0.003 ± 0.001	15.4 ± 6.8
3	39.0 ± 9.2	59.0 ± 9.2	59 ± 9	0.002 ± 0.000	14.7 ± 2.3
5	55.0 ± 8.9	75.0 ± 8.9	75 ± 9	0.003 ± 0.000	19.0 ± 2.3
10	63.0 ± 10.8	83.0 ± 10.8	166 ± 22	0.005 ± 0.001	22.0 ± 2.9
20	76.0 ± 13.9	96.0 ± 13.9	288 ± 42	0.006 ± 0.001	27.7 ± 4.0
50	65.0 ± 17.0	85.0 ± 17.0	595 ± 119	0.010 ± 0.002	30.6 ± 6.1
100	57.0 ± 6.8	77.0 ± 6.8	1001 ± 88	0.015 ± 0.001	36.9 ± 3.2
250	38.4 ± 10.5	58.4 ± 10.5	1869 ± 335	0.032 ± 0.006	105.1 ± 18.9
500	35.0 ± 22.6	55.0 ± 22.6	3465 ± 1421	0.052 ± 0.021	131.6 ± 54.0
1000	25.7 ± 8.5	45.7 ± 8.5	5708 ± 1062	0.077 ± 0.014	163.5 ± 30.4
all (1,491)	19.0 ± 10.0	39.0 ± 10.0	7293 ± 1876	0.095 ± 0.024	185.1 ± 47.6

Table 5. Experiment 2 — training cost until early stopping, random init, trainable + aug (mean ± std over seeds).

N	best epoch	stop epoch	steps	GPU hours	total TFLOP
1	34.0 ± 23.5	54.0 ± 23.5	54 ± 24	0.002 ± 0.001	13.3 ± 5.8
3	49.0 ± 28.5	69.0 ± 28.5	69 ± 29	0.003 ± 0.001	17.8 ± 7.4
5	40.0 ± 11.4	60.0 ± 11.4	60 ± 11	0.003 ± 0.001	16.1 ± 3.1
10	25.0 ± 17.6	45.0 ± 17.6	90 ± 35	0.004 ± 0.002	13.4 ± 5.2
20	35.0 ± 27.9	55.0 ± 27.9	165 ± 84	0.005 ± 0.002	19.4 ± 9.8
50	39.0 ± 5.8	59.0 ± 5.8	413 ± 41	0.010 ± 0.001	30.6 ± 3.0
100	49.0 ± 26.3	69.0 ± 26.3	897 ± 342	0.021 ± 0.008	54.9 ± 21.0
250	6.8 ± 2.0	26.8 ± 2.0	858 ± 65	0.022 ± 0.002	69.5 ± 5.3
500	11.3 ± 6.1	31.3 ± 6.1	1974 ± 386	0.046 ± 0.009	124.7 ± 24.4
1000	38.7 ± 26.1	58.7 ± 26.1	7333 ± 3260	0.162 ± 0.069	396.1 ± 176.1
all (1,491)	24.0 ± 13.0	44.0 ± 13.0	8228 ± 2424	0.173 ± 0.051	416.8 ± 122.8

GPU hours are the wall-clock time of each run (model build, training and every test check). FLOPs are counted with torch.utils.flop_counter per sample (2.37 GFLOP per training sample for the frozen encoder, which has no backward pass through the encoder; 5.54 GFLOP for the trainable one; 1.87 GFLOP per evaluated sample) and multiplied by the samples processed; the total includes every test check (644 images each).